

Day 7: Debugging from Day 1

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

Zephyr LOG Subsystem

Zephyr's logging system is the primary debugging tool. It has 4 levels, each compiled out below the configured threshold:

```
LOG_DBG("Detailed trace: val=%d", val);    // Level 4 – stripped in release
LOG_INF("Normal info: connected");         // Level 3
LOG_WRN("Warning: retry %d", count);       // Level 2
LOG_ERR("Error: init failed: %d", ret);    // Level 1 – always included
```

Each level is filtered at **compile time** (zero overhead for stripped messages) and **runtime** (can be changed per-module).

LOG_MODULE_REGISTER

Every source file that uses logging must register itself with a name and default level:

```
LOG_MODULE_REGISTER(my_module, LOG_LEVEL_DBG);
```

This allows per-module log level control via Kconfig — e.g., enable DEBUG for one driver while keeping INFO for others.

CONFIG_LOG_DEFAULT_LEVEL

In `prj.conf`:

```
CONFIG_LOG=y
CONFIG_LOG_DEFAULT_LEVEL=4    # 0=off 1=err 2=warn 3=info 4=debug
```

Set to 4 during development, 1 or 2 in production to reduce code size and output overhead.

__ASSERT() for Debug-Time Assertions

`__ASSERT(condition, "message")` checks a condition in **debug builds** and triggers a kernel oops (crash with register dump) if it fails. In release builds (`CONFIG_ASSERT=n`), the entire check is compiled out:

```
__ASSERT(buffer != nullptr, "Null buffer pointer!");
__ASSERT(len <= MAX_SIZE, "Length %d exceeds max %d", len, MAX_SIZE);
```

This is the embedded alternative to C++ exceptions — fail fast in dev, zero cost in prod.

GDB over J-Link (west debug)

```
west debug          # Launches OpenOCD + GDB, halts at main()
(gdb) break my_func # Set breakpoint
(gdb) continue      # Run until breakpoint
(gdb) print var      # Inspect variable
(gdb) backtrace      # Show call stack
(gdb) info threads   # Show Zephyr threads
```

The nRF52840DK has an on-board J-Link — no extra hardware needed.

RTT (SEGGER Real-Time Transfer)

RTT is faster than UART for logging — it uses a shared memory buffer and J-Link reads it without interrupting the CPU:

```
CONFIG_USE_SEGGER_RTT=y
CONFIG_LOG_BACKEND_RTT=y
```

View with: `JLinkRTTViewerExe` or `west attach`.

k_oops() and k_panic()

- `k_oops()` : triggers a kernel oops (soft fault) — dumps registers, stack trace. Use when an assertion fails.
- `k_panic()` : unrecoverable system failure — halts or resets depending on config.

Both print debug info via the configured log backend before stopping.

Practice Tasks

1. Add `LOG_MODULE_REGISTER(myapp, LOG_LEVEL_DBG)` and test all 4 log levels
 2. Intentionally trigger `__ASSERT(false, "test assert")` — observe the fault dump
 3. Set `CONFIG_LOG_DEFAULT_LEVEL=1` and rebuild — measure binary size reduction
 4. Use `west debug` and GDB `info threads` to list Zephyr's internal threads
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day07
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)

- [Nordic DevZone](#)
- Source: [src/day07/main.cpp](#)